

Documentation d'Installation : Architecture Web 4 Tiers (WordPress)

Objectif : Mettre en place une infrastructure web flexible et tolérante aux pannes, séparant le frontal (Reverse Proxy), le serveur applicatif (WordPress) et la base de données (MariaDB).

1. Serveur de Base de Données (BDD1)

L'objectif ici est de configurer MariaDB pour qu'il accepte les connexions venant du serveur applicatif, tout en sécurisant l'installation de base.

Sécurisation initiale

Lancez l'utilitaire de sécurisation natif de MariaDB : `mariadb-secure-installation`
Explication : Ce script interactif permet de définir un mot de passe root fort, de supprimer les utilisateurs anonymes et de désactiver les bases de données de test par défaut.

```
All done! If you've completed all of the above steps, your MariaDB
installation should now be secure.
```

Création de l'utilisateur distant

Connectez-vous à la console SQL : `mysql --user=root -p`

Exécutez les commandes suivantes pour créer l'administrateur :

SQL

```
CREATE USER 'wp_admin'@'%' IDENTIFIED BY 'MotDePasseFort';
GRANT ALL PRIVILEGES ON *.* TO 'wp_admin'@'%' WITH GRANT OPTION;
FLUSH PRIVILEGES;
```

Explication des paramètres :

- `'%'` : Ce symbole autorise l'utilisateur à se connecter depuis n'importe quelle adresse IP (et non juste en local).
- `GRANT ALL PRIVILEGES` : Donne les droits totaux pour gérer les futures bases.
- `FLUSH PRIVILEGES` : Force MariaDB à recharger la table des droits immédiatement.

```
MariaDB [mysql]> CREATE USER 'wp_admin'@'%' IDENTIFIED BY 'Etudiant_1234';
Query OK, 0 rows affected (0,001 sec)

MariaDB [mysql]> GRANT ALL PRIVILEGES ON *.* TO 'wp_admin'@'%' WITH GRANT OPTION;
Query OK, 0 rows affected (0,001 sec)

MariaDB [mysql]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0,001 sec)
```

Autorisation des connexions externes

Éditez le fichier de configuration principal : `nano /etc/mysql/mariadb.conf.d/50-server.cnf`

Modifiez la ligne `bind-address` : `bind-address = 0.0.0.0` *Explication : Par défaut, MariaDB écoute sur 127.0.0.1 (localhost). Passer en 0.0.0.0 lui ordonne d'écouter sur toutes les interfaces réseau de la machine.*

Redémarrez le service pour appliquer : `systemctl restart mariadb`

```

GNU nano 8.4 /etc/mysql/mariadb.conf.d/50-server.cnf *
#
# These groups are read by MariaDB server.
# Use it for options that only the server (but not clients) should see

# this is read by the standalone daemon and embedded servers
[server]

# this is only for the mariadb daemon
[mariadb]

#
# * Basic Settings
#

#user                = mysql
pid-file             = /run/mysqld/mysqld.pid
basedir              = /usr
#datadir              = /var/lib/mysql
#tmpdir               = /tmp

# Broken reverse DNS slows down connections considerably and name resolve is
# safe to skip if there are no "host by domain name" access grants
#skip-name-resolve

# Instead of skip-networking the default is now to listen only on
# localhost which is more compatible and is not less secure.
bind-address          = 0.0.0.0

#
# * Fine Tuning
#

#key_buffer_size      = 128M
#max_allowed_packet    = 1G
#thread_stack          = 192K
#thread_cache_size     = 8
# This replaces the startup script and checks MyISAM tables if needed
# the first time they are touched
#myisam_recover_options = BACKUP
#max_connections       = 100
#table_cache            = 64

```

2. Serveur Web Applicatif (WP1)

Ce serveur va héberger les fichiers WordPress et exécuter le code PHP. Il ne sera pas exposé directement sur Internet.

Test de communication BDD

Installez le client MariaDB pour tester l'accès distant : `apt install mariadb-client`
`mysql -u wp_admin -p -h 192.168.160.31 -P 3306` Explication : `-h` spécifie l'hôte cible (l'IP de BDD1) et `-P` le port standard de MariaDB.

```
root@mod-debian13:~# mysql -u wp_admin -p -h 192.168.160.31 -P 3306
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 31
Server version: 11.8.3-MariaDB-0+deb13u1 from Debian -- Please help get to 10k stars at
MariaDB/Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]>
MariaDB [(none)]>
```

```
MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| phpmyadmin |
| sys |
+-----+
5 rows in set (0,002 sec)
```

Installation de l'environnement (Nginx & PHP)

Installez le serveur web et les modules PHP requis par WordPress : `apt update && apt upgrade apt install nginx php php-fpm php-mysqlnd php-curl php-gd php-mbstring php-xml php-intl php-zip` *Explication : Nginx sert les pages web. `php-fpm` est le gestionnaire de processus PHP pour Nginx. Les autres paquets sont des extensions nécessaires au bon fonctionnement de WordPress (gestion d'images, de bases de données, etc.).*

Vérifiez que Nginx tourne : `service nginx status`

```

root@mod-debian13:~# service nginx status
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-03-18 09:27:03 CET; 1h 52min ago
 Invocation: 77a100efb1cb4856b670cd02d9c3bee2
    Docs: man:nginx(8)
   Process: 861 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=
   Process: 873 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, s
 Main PID: 888 (nginx)
    Tasks: 3 (limit: 2255)
   Memory: 5.3M (peak: 5.5M)
      CPU: 26ms
   CGroup: /system.slice/nginx.service
           └─888 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─889 "nginx: worker process"
               └─890 "nginx: worker process"

```

Préparation de la base de données WordPress

Depuis WP1 (ou BDD1), connectez-vous à MariaDB et créez la base dédiée :

```
CREATE DATABASE wordpress_db DEFAULT CHARACTER SET utf8 COLLATE
utf8_unicode_ci;
```

Installation de WordPress

Téléchargez et extrayez l'archive dans le dossier web :

```

cd /tmp
wget https://fr.wordpress.org/latest-fr_FR.zip
unzip latest-fr_FR.zip
mv wordpress /var/www/html/

```

1.

Configuration des accès BDD :

```

cd /var/www/html/wordpress
cp wp-config-sample.php wp-config.php
nano wp-config.php

```

2. Renseignez le nom de la base, l'utilisateur (*wp_admin*) et l'IP de BDD1.
3. Attribution des droits : `chown -R www-data:www-data /var/www/html/wordpress` Explication : *www-data* est l'utilisateur sous lequel Nginx s'exécute. Il doit être propriétaire des fichiers pour pouvoir les lire et les modifier (ex: upload d'images).

```
GNU nano 8.4 wp-config.php
<?php
/**
 * The base configuration for WordPress
 *
 * The wp-config.php creation script uses this file during the installation.
 * You don't have to use the website, you can copy this file to "wp-config.php"
 * and fill in the values.
 *
 * This file contains the following configurations:
 *
 * * Database settings
 * * Secret keys
 * * Database table prefix
 * * ABSPATH
 *
 * @link https://developer.wordpress.org/advanced-administration/wordpress/wp-config/
 *
 * @package WordPress
 */

// ** Database settings - You can get this info from your web host ** //
/** The name of the database for WordPress */
define( 'DB_NAME', 'wordpress_db' );

/** Database username */
define( 'DB_USER', 'wp_admin' );

/** Database password */
define( 'DB_PASSWORD', 'Etudiant_1234' );

/** Database hostname */
define( 'DB_HOST', '192.168.160.31' );

/** Database charset to use in creating database tables. */
define( 'DB_CHARSET', 'utf8mb4' );

/** The database collate type. Don't change this if in doubt. */
define( 'DB_COLLATE', '' );

/**#@+
```

Configuration de Nginx pour WordPress

Éditez le fichier par défaut : `nano /etc/nginx/sites-enabled/default`

Modifiez pour cibler WordPress et interpréter le PHP :

Nginx

```
server {
    listen 80 default_server;
    root /var/www/html/wordpress;
    index index.php index.html;

    location / {
        try_files $uri $uri /index.php?$args;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
```

```
    fastcgi_pass unix:/run/php/php-fpm.sock;
}
}
```

Redémarrez Nginx : `service nginx restart`.

```
# Default server configuration
#
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    # SSL configuration
    #
    # listen 443 ssl default_server;
    # listen [::]:443 ssl default_server;
    #
    # Note: You should disable gzip for SSL traffic.
    # See: https://bugs.debian.org/773332
    #
    # Read up on ssl_ciphers to ensure a secure configuration.
    # See: https://bugs.debian.org/765782
    #
    # Self signed certs generated by the ssl-cert package
    # Don't use them in a production server!
    #
    # include snippets/snakeoil.conf;

    root /var/www/html/wordpress;

    # Add index.php to the list if you are using PHP
    index index.html index.php;

    server_name _;

    location / {
        # First attempt to serve request as file, then
        # as directory, then fall back to displaying a 404.
        try_files $uri $uri/ /index.php?args;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php-fpm.sock;
    }
}
```

3. Serveur Frontal (FRONT1)

Ce serveur est le seul exposé à Internet. Il agit comme un proxy inverse (Reverse Proxy) et gère la sécurité SSL.

Configuration du Reverse Proxy

Installez Nginx (`apt install nginx`) et créez le fichier de routage : `nano /etc/nginx/sites-available/wordpress`

Insérez la configuration fournie dans les consignes (le bloc `upstream wordpress-srv et server`). *Explication : `proxy_pass` indique à Nginx de transférer toutes les requêtes HTTP entrantes vers l'IP de notre serveur WP1 (192.168.160.21) de manière transparente pour le client.*

Activez la configuration : `ln -s /etc/nginx/sites-available/wordpress /etc/nginx/sites-enabled/ service nginx configtest && service nginx restart`

Mise en place du SSL (Auto-signé)

Générez les clés de chiffrement :

```
mkdir /etc/nginx/ssl
```

```
openssl req -x509 -nodes -days 3000 -newkey rsa:2048 -keyout  
/etc/nginx/ssl/front.tp.labo.key -out  
/etc/nginx/ssl/front.tp.labo.crt
```

Explication des paramètres :

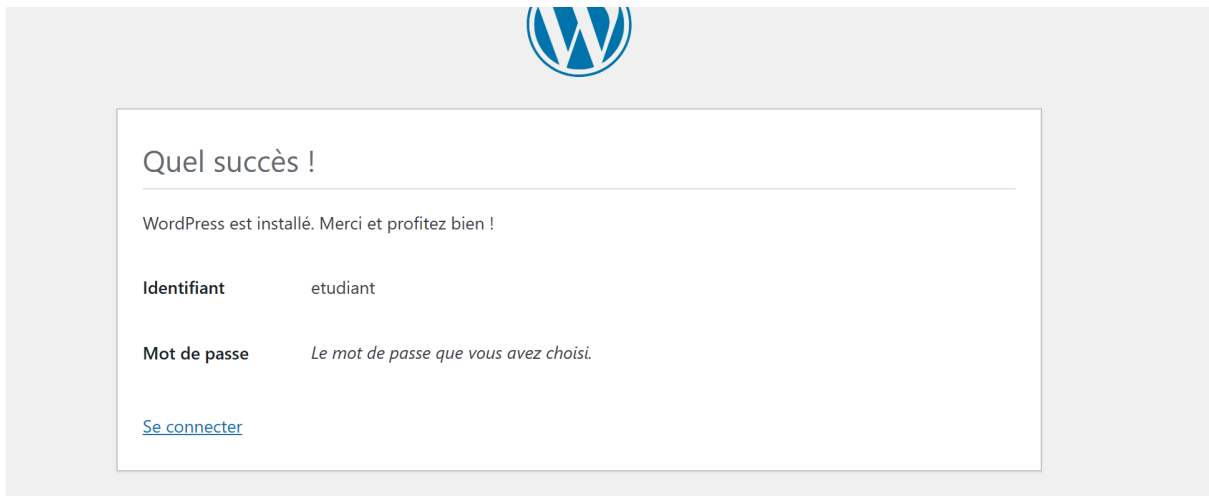
- `-x509` : Crée un certificat auto-signé au lieu d'une simple demande.
- `-nodes` : Pas de mot de passe sur la clé privée (nécessaire pour que Nginx redémarre tout seul).
- `-days 3000` : Durée de validité.
- `-newkey rsa:2048` : Génère une nouvelle clé RSA robuste.

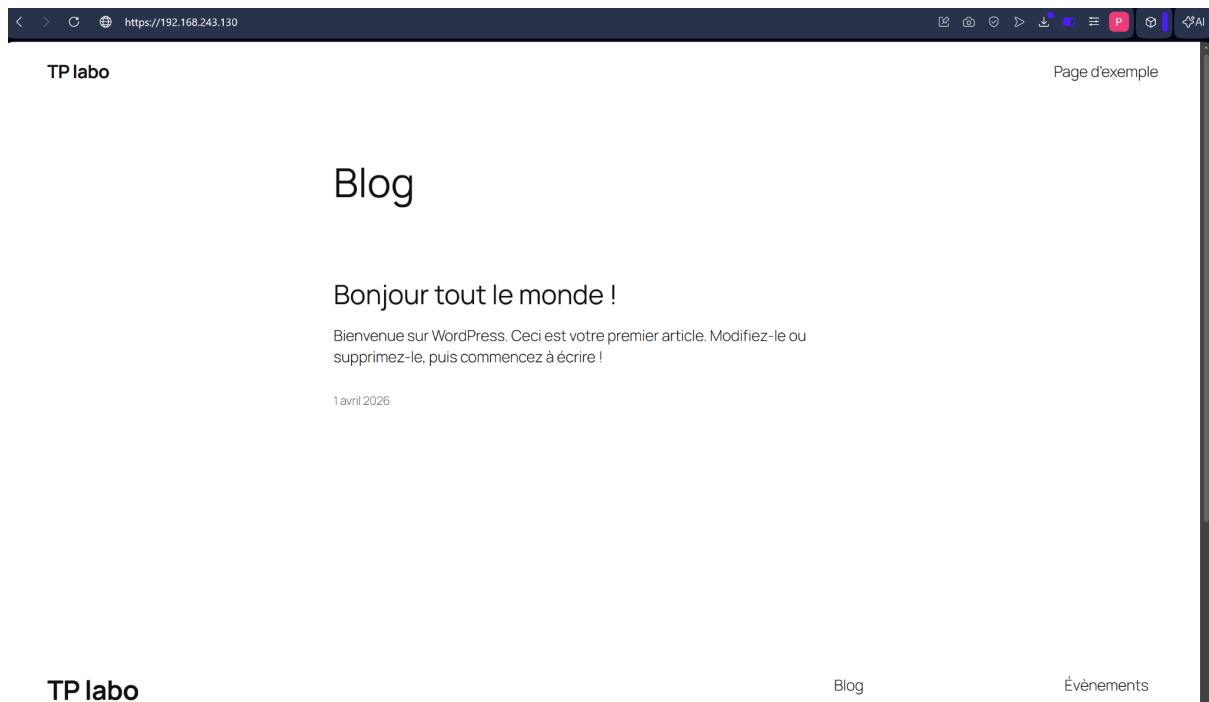
Modifiez le fichier `/etc/nginx/sites-available/wordpress` pour ajouter l'écoute sur le port `443` (HTTPS) et la redirection du port `80` vers `443`, comme indiqué dans les consignes.

Correction du problème de CSS (Boucle de redirection HTTPS)

Pour régler le problème d'affichage (WordPress croit être en HTTP alors que le proxy est en HTTPS), éditez le fichier `/var/www/html/wordpress/wp-config.php` sur le serveur **WP1**. Ajoutez ceci juste avant la ligne:

```
if (isset($_SERVER['HTTP_X_FORWARDED_PROTO']) &&
$_SERVER['HTTP_X_FORWARDED_PROTO'] === 'https') {
    $_SERVER['HTTPS'] = 'on';
}
```





4. Administration : PhpMyAdmin (Sur le Front)

Pour faciliter la gestion de la BDD, PhpMyAdmin est installé sur le Frontal.

`apt install phpmyadmin php-fpm php-mbstring php-mysql ...` Lors de l'installation, refusez la configuration automatique de la base locale.

Configuration pour la base distante

Éditez le fichier de configuration de PhpMyAdmin : `nano /etc/phpmyadmin/config.inc.php` Trouvez la section des serveurs et pointez vers l'IP de BDD1 : `$cfg['Servers'][$i]['host'] = '192.168.160.31';`

Configuration Nginx (Port 4443)

Ajoutez un nouveau bloc `server` dans votre configuration Nginx (`/etc/nginx/sites-available/wordpress`):

```
Nginx
server {
    listen 4443 ssl;
    server_name front.tp.labo;
    root /usr/share/phpmyadmin;
    index index.php index.html;

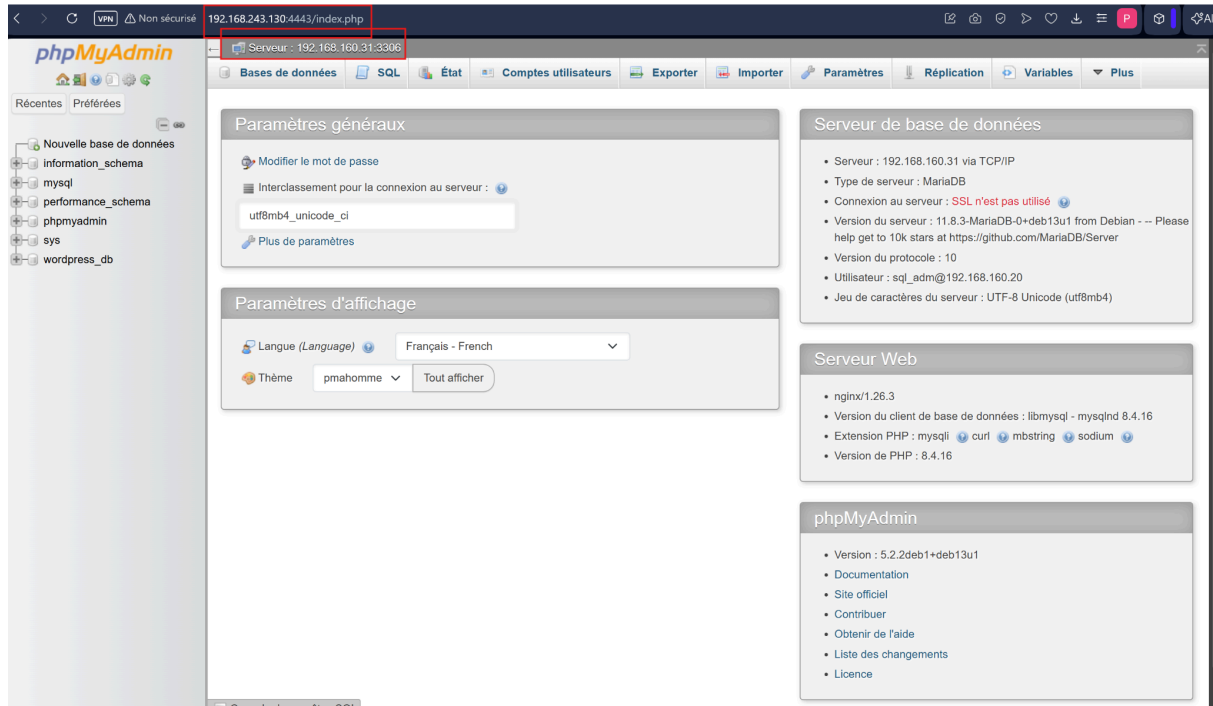
    ssl_certificate /etc/nginx/ssl/front.tp.labo.crt;
    ssl_certificate_key /etc/nginx/ssl/front.tp.labo.key;
```

```

location ~ \.php$ {
    include snippets/fastcgi-php.conf;
    fastcgi_pass unix:/run/php/php-fpm.sock;
}
}

```

Redémarrez Nginx. L'interface est désormais accessible via https://IP_FRONT:4443.



Documentation Technique : Déploiement de la Haute Disponibilité (Architecture 4 Tiers)

1. Mise à niveau de l'infrastructure (Upgrade)

Objectif et Contexte

*Pour transformer notre architecture 4 tiers initiale en une infrastructure à **Haute Disponibilité (HA)**, nous avons doublé les composants critiques (serveurs applicatifs et bases de données). L'objectif est d'éliminer les points de défaillance uniques (Single Point of Failure) : si un serveur tombe, le second prend le relais de manière transparente.*

Méthodologie de déploiement

Le déploiement des nouveaux nœuds a été réalisé par clonage afin de garantir une parfaite parité logicielle et configurationnelle entre les serveurs.

1. **Sécurisation de l'existant** : Avant toute modification, les machines WP1 et BDD1 ont été arrêtées proprement et un "Snapshot" (instantané) de sécurité a été réalisé sous l'hyperviseur.
2. **Clonage intégral** : Création des machines **WP2** (clone de WP1) et **BDD2** (clone de BDD1) via l'outil de clonage.
3. **Note sur l'identité des hôtes** : Par choix technique dans ce laboratoire, les noms d'hôtes (hostnames) ont été conservés identiques aux originaux pour simplifier le déploiement. L'identification et le routage se font exclusivement par l'adressage IP statique.

Configuration réseau initiale des nouveaux serveurs

*Chaque nouveau clone a été configuré avec une adresse IP statique propre dans le segment LAN isolé, en utilisant le serveur **FRONT1 (192.168.160.20)** comme passerelle par défaut.*

Commandes de configuration (réalisées sur WP2 et BDD2) :

L'édition du fichier de configuration réseau est indispensable au premier démarrage pour éviter les conflits d'IP avec les machines sources :

```
nano /etc/network/interfaces
```

Configuration appliquée pour WP2 :

- `address 192.168.160.22`

- `netmask 255.255.255.0`
- `gateway 192.168.160.20`

Configuration appliquée pour BDD2 :

- `address 192.168.160.32`
- `netmask 255.255.255.0`
- `gateway 192.168.160.20`

Vérification de la connectivité

Après redémarrage des services réseau, la validation est effectuée via deux tests majeurs depuis les nouveaux serveurs :

1. **Vérification de l'interface locale** : Utilisation de la commande `ip a` pour confirmer la bonne attribution des adresses IP statiques (.22 et .32).
2. **Test de sortie réseau** : Utilisation de la commande `ping 8.8.8.8` pour confirmer que le routage à travers le pare-feu/passarelle FRONT1 est pleinement opérationnel.

```
etudiant@mod-debian13:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:af:cd:2f brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c29afcd2f
    inet 192.168.160.22/24 brd 192.168.160.255 scope global ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:feaf:cd2f/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
```

```
etudiant@mod-debian13:/etc/network$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=127 time=42.3 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=127 time=51.1 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=127 time=40.9 ms
```

```
etudiant@mod-debian13:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:70:34:ff brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c297034ff
    inet 192.168.160.32/24 brd 192.168.160.255 scope global ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe70:34ff/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
```

```
etudiant@mod-debian13:~$ ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=127 time=35.5 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=127 time=166 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=127 time=147 ms
```

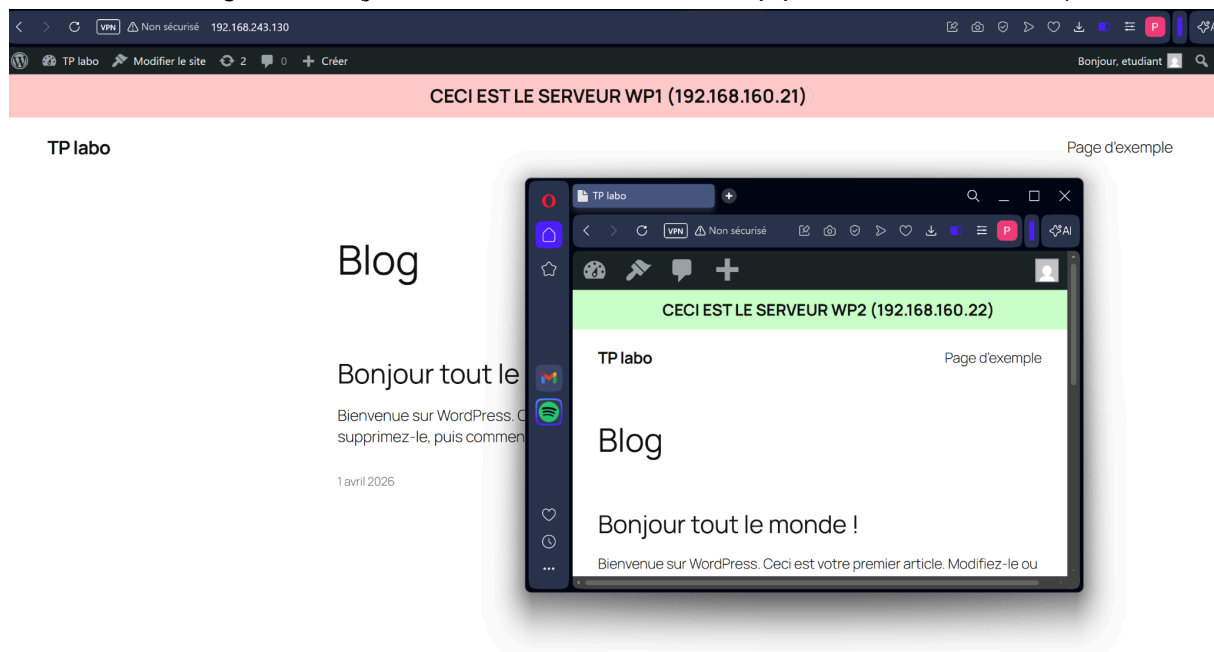
2. Configuration du Load Balancing (Répartition de charge)

Mise en place d'un répartiteur de charge NGINX sur le(s) serveur(s) Frontal(aux) pour distribuer le trafic web.

- **Méthode Round Robin** : Distribution égale des requêtes.

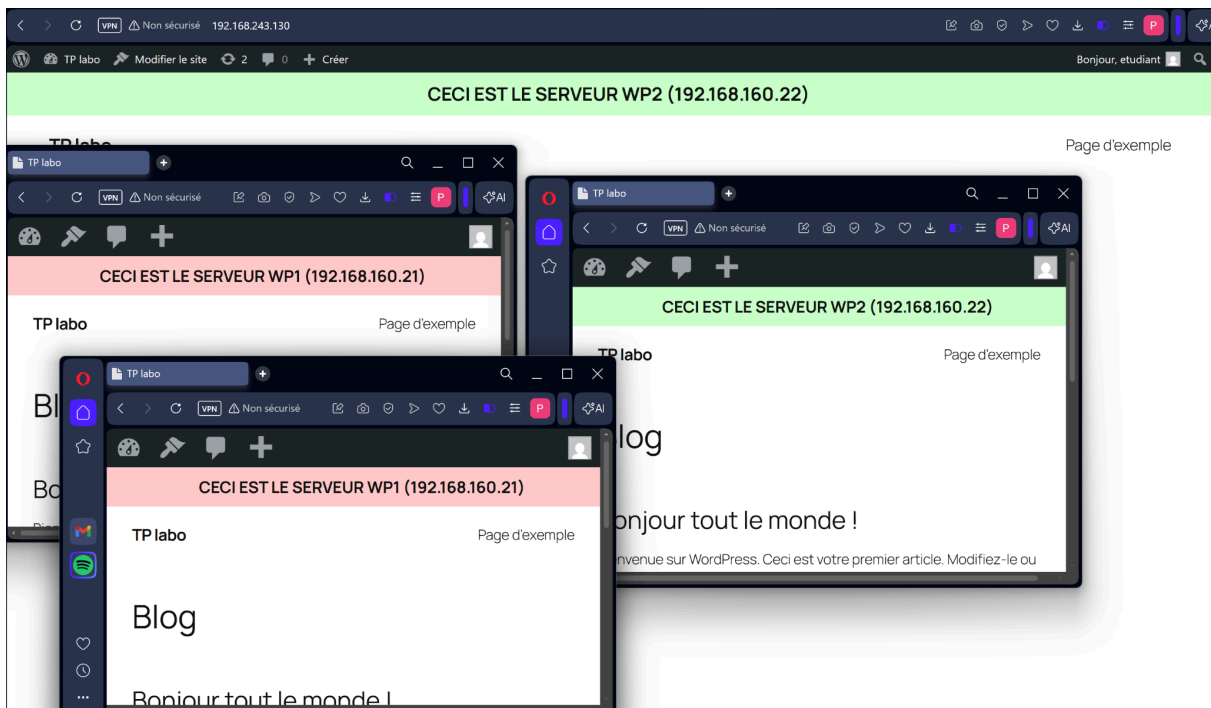
```
upstream wordpress-srv {
    server 192.168.160.21;
    server 192.168.160.22;
}
```

Le load Balancing est actif (j'ai modifier les headers des wp pour voir la différence):



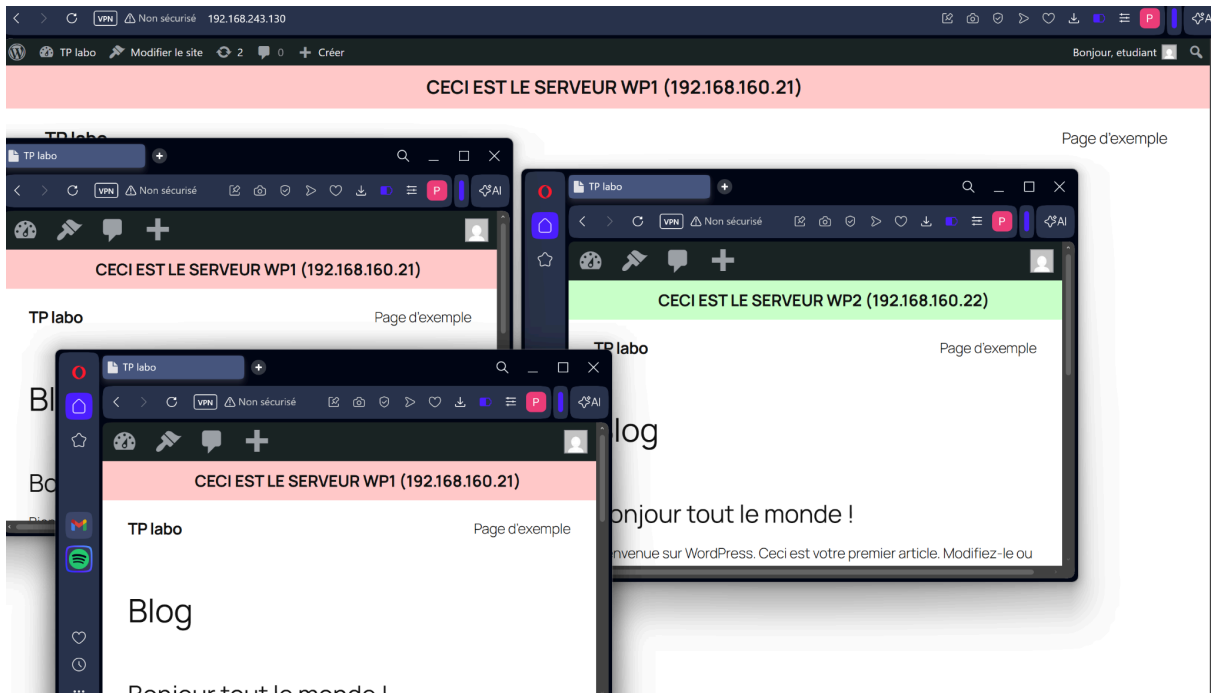
- **Méthode Server Weights** : Une requête est envoyée au serveur avec le moins de connexions actives.

```
upstream wordpress-srv {  
    least_conn;  
    server 192.168.160.21;  
    server 192.168.160.22;  
}
```



- **Méthode Server weights** : Définir un poids fixe

```
upstream wordpress-srv {  
    server 192.168.160.21 weight=3;  
    server 192.168.160.22 weight=1;  
}
```



- **Méthode IP hash :** Sépare les connexions avec l'ip du client.

```
upstream wordpress-srv {
    ip_hash;
    server 192.168.160.21;
    server 192.168.160.22;
}
```

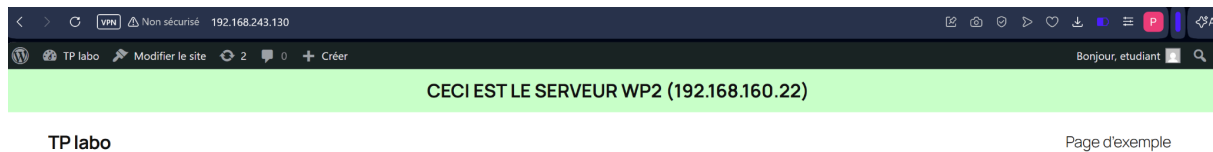
L'ip de ma machine étant en .1 je reste uniquement sur WP1.



Maintenant je reprend la conf **Round Robin**, on va éteindre wp1 et on va s'assurer que ça prend la relève sur wp2.

```
root@mod-debian13:~# tail /var/log/nginx/wordpress.error.log
2026/05/13 09:30:12 [error] 4715#4715: *36 connect() failed (113: No route to host) while connecting to upstream, client: 192.168.243.1, server: front.tp.labo, request: "GET /wp-includes/js/dist/script-modules/block-library/navigation/view.min.js?ver=b0f909c3ec791c383210 HTTP/1.1", upstream: "http://192.168.160.21:80/wp-includes/js/dist/script-modules/block-library/navigation/view.min.js?ver=b0f909c3ec791c383210", host: "192.168.243.130", referer: "https://192.168.243.130/"
2026/05/13 09:30:12 [error] 4714#4714: *32 connect() failed (113: No route to host) while connecting to upstream, client: 192.168.243.1, server: front.tp.labo, request: "GET /wp-includes/js/dist/script-modules/interactivity/index.min.js?ver=66c613f68580994bb00a HTTP/1.1", upstream: "http://192.168.160.21:80/wp-includes/js/dist/script-modules/interactivity/index.min.js?ver=66c613f68580994bb00a", host: "192.168.243.130", referer: "https://192.168.243.130/"
```

Il ne trouve pas WP1, ça crée une erreur et il redirige vers WP2 uniquement.



Blog

Bonjour tout le monde !

Bienvenue sur WordPress. Ceci est votre premier article. Modifiez-le ou supprimez-le, puis commencez à écrire !

1 avril 2026

3. Synchronisation des serveurs Web (Persistance des données)

Problématique : L'incohérence des fichiers médias

Dans une architecture équilibrée (Load Balancing), un utilisateur qui téléverse un fichier (ex: une image WordPress) via le serveur **WP1** ne la verra pas s'il est redirigé sur **WP2** lors de sa visite suivante (Erreur 404). Il est donc impératif que les dossiers `/var/www/html/wordpress` soient synchronisés.

Solution technique : Unison

Pour réaliser une synchronisation bi-directionnelle, nous avons déployé l'outil **Unison**. Contrairement à un simple transfert unidirectionnel, Unison compare les deux répertoires et propage les modifications dans les deux sens, garantissant une cohérence totale.

Étapes de mise en œuvre :

1. **Autorisation de connexion SSH pour root** : Modification du fichier `/etc/ssh/sshd_config` (`PermitRootLogin yes`) sur les deux serveurs pour permettre l'accès réseau du compte administrateur.
2. **Création et échange des clés SSH** : Sur WP1, génération d'une paire de clés privée/publique (`ssh-keygen`) et transfert de la clé publique vers WP2 (`ssh-copy-id`) pour permettre l'ouverture de session sans mot de passe.
3. **Installation des prérequis** : Installation des paquets nécessaires sur WP1 et WP2 via la commande `apt install unison ssh`.
4. **Exécution de la synchronisation** : Lancement de la commande Unison pour lier les répertoires applicatifs des deux nœuds.

Voici la clé présente sur WP2 après le transfert de la clé ssh:

```
root@mod-debian13:~# cat /root/.ssh/authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQgC53fU100bR+1NsrdRjWt3pfBGnp67jiic60679hbbnFqJ45X0d7P5MocMFEUrsJAE6uRPK0njkPKetITL+Uv3/vMW1mLd0SGGA+gPz+10RRkfwpgar3e
wGsoYTK+8275vBhoal+FC9Ar0DEoSVMMAK+0/d6vrm6gplUC80Zru18gm7UppVCamoQqr28J0CP+V7p9EbwMLe8Lmw0TkunnGjku25D8rg1YrKFjPdVKEJu0mjwT5H2eThfP0qDehwQ0fwLLYrAjdTmxHf
Z+1Zn1Pk2W94ATonjyx1pNM5B12Sjot3J3sW0VneIF4xeYbw5xXfr0SzcJ+B5D9s0TbydC656R5T01x1d50TQA0ZIpUM5qTah4eqFmMbygk0ng6hQ+Mqaf3aDzqFQ9gbnZTFIja7K6clp1nvp1V1chBrf4V
EtdUrwzUOK6JGonJ38PRZzyL1J38rdbazLFzPRZd0iLUDmPjyQCugm10ILOeFDCCuQgthrKugm0N7P119IXj5+c29ns= root@mod-debian13
```

Sur WP1 on peut lancer la connexion a WP2 sans avoir à mettre de mdp:

```
root@mod-debian13:~# ssh root@192.168.160.22
Linux mod-debian13 6.12.74+deb13+1-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.12.74-2 (2026-03-08) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed May 13 11:57:39 2026 from 192.168.160.21
root@mod-debian13:~#
```

On download une image sur WP1:

```
root@mod-debian13:~# mkdir -p /var/www/html/wordpress/wp-content/uploads
cd /var/www/html/wordpress/wp-content/uploads
wget https://cours-informatique-gratuit.fr/wp-content/uploads/2018/06/synchronisation.png
--2026-05-25 10:56:42-- https://cours-informatique-gratuit.fr/wp-content/uploads/2018/06/synchronisation.png
Résolution de cours-informatique-gratuit.fr (cours-informatique-gratuit.fr)... 109.234.166.36
Connexion à cours-informatique-gratuit.fr (cours-informatique-gratuit.fr) [109.234.166.36]:443... connecté.
requête HTTP transmise, en attente de la réponse... 200 OK
Taille : 12253 (12K) [image/png]
Sauvegarde en : « synchronisation.png »

synchronisation.png 100%[=====] 11,97K --.-KB/s ds 0s
2026-05-25 10:56:42 (105 MB/s) - « synchronisation.png » sauvegardé [12253/12253]
root@mod-debian13:/var/www/html/wordpress/wp-content/uploads# ls -lh synchronisation.png
-rw-r--r-- 1 root root 12K 18 juin 2018 synchronisation.png
```

On lance la commande Unison et on vérifie que l'image synchronisation.png soit présente:

```

root@mod-debian13:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:af:cd:2f brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c29afcd2f
    inet 192.168.160.22/24 brd 192.168.160.255 scope global ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:feaf:cd2f/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
root@mod-debian13:~# ls -lh /var/www/html/wordpress/wp-content/uploads/synchronisation.png
-rw-r--r-- 1 root root 12K 18 juin 2018 /var/www/html/wordpress/wp-content/uploads/synchronisation.png

```

L'image est bien présente sur WP2 donc bien synchro.

Automatisation de la synchronisation via Unison et Cron

- **Fichier de profil (/root/.unison/default.prf)** : Pour automatiser le processus, un profil de configuration a été créé. Les paramètres clés configurés sont :
 - **root** : Définition des répertoires source et destination.
 - **auto = true** et **batch = true** : Activation du mode autonome sans invite de commande.
 - **prefer = newer** : Règle de gestion des conflits en faveur du fichier le plus récent.
 - **localhostname** : Forçage de l'identité du nœud pour contourner la collision des verrous due au clonage des machines.
- **Planification (crontab)** : Ajout d'une règle d'exécution récurrente (* / 2 * * * *) pour déclencher le démon Unison de manière transparente toutes les deux minutes.

Fichier de profil :

```

GNU nano 8.4 /root/.unison/default.prf *
# Unison preferences file
# --- Chemins des répertoires à synchroniser ---
root = /var/www/html/wordpress
root = ssh://192.168.160.22/var/www/html/wordpress

# --- Paramètres d'automatisation ---
# Accepte automatiquement les modifications non conflictuelles
auto = true
# S'exécute en mode batch (sans intervention humaine, pas de question)
batch = true

# --- Sécurités et gestion des conflits ---
# En cas de conflit (modification simultanée), la version la plus récente gagne
prefer = newer
# Résout le bug de verrouillage dû au clonage des VM (noms d'hôtes identiques)
localhostname = ServeurWP1

```

crontab config pour qu'il se lance toute les 2 minutes :

```
GNU nano 8.4 /tmp/crontab.dLban4/crontab *
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow command
*/2 * * * * UNISONLOCALHOSTNAME=ServeurWP1 /usr/bin/unison > /dev/null 2>&1
```

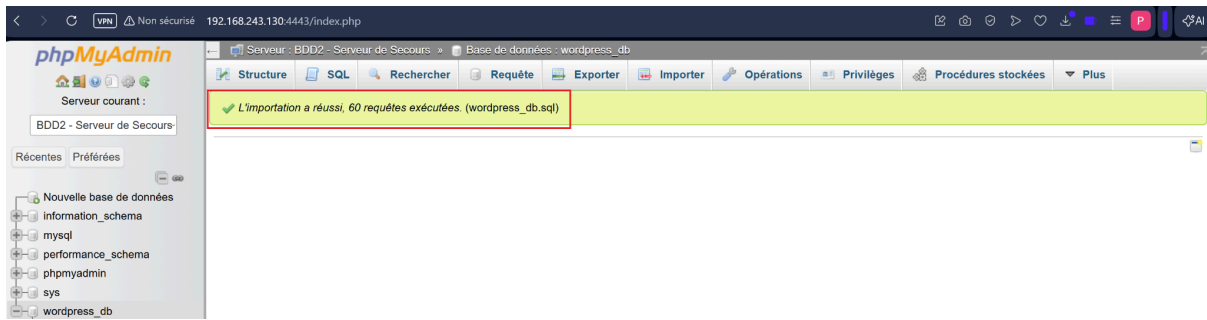
crontab: installing new crontab

4. Réplication de la base de données MariaDB

Objectif : Haute disponibilité et intégrité des données Le serveur de base de données est le composant le plus critique de l'infrastructure. Pour pallier la défaillance d'un nœud et éviter d'avoir un jeu de données désynchronisé, nous mettons en place une réplication **Maître-Maître (Bidirectionnelle)**. Chaque serveur (BDD1 et BDD2) assure les opérations de lecture et d'écriture tout en enregistrant les modifications de l'autre.

1. Préparation et mise à niveau des bases de données Avant de démarrer la synchronisation, il est impératif d'avoir deux bases de données rigoureusement identiques.

- **Configuration de PhpMyAdmin** : Ajout des deux instances (BDD1 et BDD2) dans le fichier `/etc/phpmyadmin/config.inc.php` du serveur frontal pour permettre une administration centralisée via un menu déroulant de sélection de serveur.
- **Sauvegarde (Export)** : Depuis PhpMyAdmin, export complet de la base de données `wordpress` du serveur maître (BDD1) au format `.sql`.
- **Restauration (Import)** : Sur le serveur de secours (BDD2), suppression préalable de toutes les tables de la base de données existante. Ensuite, importation du fichier `.sql` généré par BDD1 (*Note : en cas d'erreur due à la taille du fichier imposée par PHP, l'importation peut être forcée en ligne de commande*).*



2. Configuration du Nœud 1 (BDD1 : 192.168.160.31)

- **Fichier de configuration** (`/etc/mysql/mariadb.conf.d/50-server.cnf`) :
Activation et configuration stricte de la journalisation.

`server-id = 1`

`log_bin = /var/log/mysql/mysql-bin.log`

`log_bin_index = /var/log/mysql/mysql-bin.index`

`binlog_format = mixed`

`binlog_do_db = wordpress`

```
GNU nano 8.4 /etc/mysql/mariadb.conf.d/50-server.cnf
#log_slow_query_time      = 10
#log_slow_verbosity       = query_plan,explain
#log-queries-not-using-indexes
#log_slow_min_examined_row_limit = 1000

# The following can be used as easy to replay backup logs or for replication.
# note: if you are setting up a replica, see README.Debian about other
# settings you may need to change.
server-id = 1
expire_logs_days = 10
log_bin = /var/log/mysql/mysql-bin.log
log_bin_index = /var/log/mysql/mysql-bin.index
binlog_format = mixed
binlog_do_db = wordpress
#max_binlog_size          = 100M
```

- **Prérequis système** : Création du répertoire de logs (`mkdir -p /var/log/mysql`), attribution des droits à l'utilisateur `mysql`, et redémarrage du service MariaDB.
- **Action (Rôle Maître)** : Création de l'utilisateur dédié à la réplication via la requête :
`GRANT REPLICATION SLAVE ON *.* TO 'wp_replication'@'192.168.160.32' IDENTIFIED BY 'Iut_2025';`
- **Vérification** : Récupération des coordonnées exactes du journal (File et Position) via la commande `SHOW MASTER STATUS;`

```

MariaDB [(none)]> GRANT REPLICATION SLAVE ON *.* TO 'wp_replication'@'192.168.160.32' IDENTIFIED BY 'Iut_2025';
Query OK, 0 rows affected (0,002 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0,001 sec)

MariaDB [(none)]> SHOW MASTER STATUS;
+-----+-----+-----+-----+
| File           | Position | Binlog_Do_DB | Binlog_Ignore_DB |
+-----+-----+-----+-----+
| mysql-bin.000002 |      342 | wordpress    |                    |
+-----+-----+-----+-----+
1 row in set (0,000 sec)

```

3. Configuration du Nœud 2 (BDD2 : 192.168.160.32)

- **Fichier de configuration** : Configuration identique à BDD1, mais avec un identifiant unique distinct.
- Plaintext

server-id = 2

log_bin = /var/log/mysql/mysql-bin.log

log_bin_index = /var/log/mysql/mysql-bin.index

binlog_format = mixed

binlog_do_db = wordpress

```

GNU nano 8.4 /etc/mysql/mariadb.conf.d/50-server.cnf
#log_slow_query_time      = 10
#log_slow_verbosity       = query_plan,explain
#log-queries-not-using-indexes
#log_slow_min_examined_row_limit = 1000

# The following can be used as easy to replay backup logs or for replication.
# note: if you are setting up a replica, see README.Debian about other
#      settings you may need to change.
server-id = 2
expire_logs_days = 10
log_bin = /var/log/mysql/mysql-bin.log
log_bin_index = /var/log/mysql/mysql-bin.index
binlog_format = mixed
binlog_do_db = wordpress
#max_binlog_size          = 100M

```

- **Action (Rôle Esclave)** : Liaison au premier nœud via la commande SQL **CHANGE MASTER TO...** en utilisant l'utilisateur **wp_replication** et la position exacte du log binaire de BDD1. Activation via **START SLAVE;**.
- **Action (Rôle Maître)** : Pour permettre la réplication dans l'autre sens, création de l'utilisateur de réplication sur BDD2 autorisant l'IP de BDD1, et récupération de ses coordonnées (**SHOW MASTER STATUS;**).

```

MariaDB [(none)]> CHANGE MASTER TO
-> MASTER_HOST = '192.168.160.31',
-> MASTER_USER = 'wp_replication',
-> MASTER_PASSWORD = 'Iut_2025',
-> MASTER_LOG_FILE = 'mysql-bin.000002',
-> MASTER_LOG_POS = 342;
Query OK, 0 rows affected, 1 warning (0,050 sec)

MariaDB [(none)]>
MariaDB [(none)]> START SLAVE;
Query OK, 0 rows affected (0,001 sec)

MariaDB [(none)]> GRANT REPLICATION SLAVE ON *.* TO 'wp_replication'@'192.168.160.31' IDENTIFIED BY 'Iut_2025';
Query OK, 0 rows affected (0,002 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0,001 sec)

MariaDB [(none)]> SHOW MASTER STATUS;
+-----+-----+-----+-----+
| File           | Position | Binlog_Do_DB | Binlog_Ignore_DB |
+-----+-----+-----+-----+
| mysql-bin.000002 |      342 | wordpress    |                   |
+-----+-----+-----+-----+
1 row in set (0,001 sec)

```

4. Finalisation de la boucle (Sur BDD1) Liaison de BDD1 vers BDD2 via la commande **CHANGE MASTER TO...** en utilisant les coordonnées du log binaire de BDD2, suivie de **START SLAVE**; . La réplication croisée est désormais active.

5. Vérification de l'état Pour garantir le bon fonctionnement de l'architecture, l'état de la réplication doit être contrôlé sur les deux serveurs à l'aide de la commande : **SHOW SLAVE STATUS\G**

Le succès de la mise en place est confirmé lorsque les variables suivantes sont validées sur les deux machines :

- **Slave_IO_Running: Yes** (Le serveur communique bien avec l'autre nœud)
- **Slave_SQL_Running: Yes** (Le serveur parvient à écrire les données reçues)

Slave Status BDD1 :

```

MariaDB [(none)]> SHOW SLAVE STATUS\G
***** 1. row *****
      Slave_IO_State: Waiting for master to send event
        Master_Host: 192.168.160.32
        Master_User: replica_user
        Master_Port: 3306
        Connect_Retry: 60
        Master_Log_File: mysql-bin.000001
      Read_Master_Log_Pos: 660
        Relay_Log_File: mysqld-relay-bin.000002
        Relay_Log_Pos: 555
      Relay_Master_Log_File: mysql-bin.000001
      Slave_IO_Running: Yes
      Slave_SQL_Running: Yes
        Replicate_Do_DB:
      Replicate_Ignore_DB:
      Replicate_Do_Table:
    Replicate_Ignore_Table:
    Replicate_Wild_Do_Table:
    Replicate_Wild_Ignore_Table:
          Last_Errno: 0
          Last_Error:
        Skip_Counter: 0
      Exec_Master_Log_Pos: 660
        Relay_Log_Space: 865
        Until_Condition: None
        Until_Log_File:
        Until_Log_Pos: 0
      Master_SSL_Allowed: Yes
      Master_SSL_CA_File:
      Master_SSL_CA_Path:
      Master_SSL_Cert:
      Master_SSL_Cipher:
      Master_SSL_Key:
    Seconds_Behind_Master: 0
Master_SSL_Verify_Server_Cert: Yes
          Last_IO_Errno: 0
          Last_IO_Error:

```

Slave Status BBD2 :

```

MariaDB [(none)]> SHOW SLAVE STATUS\G
***** 1. row *****
      Slave_IO_State: Waiting for master to send event
        Master_Host: 192.168.160.31
        Master_User: replica_user
        Master_Port: 3306
        Connect_Retry: 60
        Master_Log_File: mysql-bin.000001
        Read_Master_Log_Pos: 660
        Relay_Log_File: mysqld-relay-bin.000002
        Relay_Log_Pos: 555
        Relay_Master_Log_File: mysql-bin.000001
        Slave_IO_Running: Yes
        Slave_SQL_Running: Yes
        Replicate_Do_DB:
        Replicate_Ignore_DB:
        Replicate_Do_Table:
        Replicate_Ignore_Table:
        Replicate_Wild_Do_Table:
        Replicate_Wild_Ignore_Table:
          Last_Errno: 0
          Last_Error:
          Skip_Counter: 0
        Exec_Master_Log_Pos: 660
        Relay_Log_Space: 865
        Until_Condition: None
        Until_Log_File:
        Until_Log_Pos: 0
        Master_SSL_Allowed: Yes
        Master_SSL_CA_File:
        Master_SSL_CA_Path:
        Master_SSL_Cert:
        Master_SSL_Cipher:
        Master_SSL_Key:
        Seconds_Behind_Master: 0
Master_SSL_Verify_Server_Cert: Yes
          Last_IO_Errno: 0
          Last_IO_Error:
          Last_SQL_Errno: 0
          Last_SQL_Error:
        Replicate_Ignore_Server_Ids:

```

On va s'assurer que cela fonctionne, dans un premier temps on stop nginx sur l'un de nos serveurs web pour ne pas qu'on perde nos actions sur l'interface et on crée un article:

Test syncro

Écrit par [etudiant](#) dans [Non classé](#)



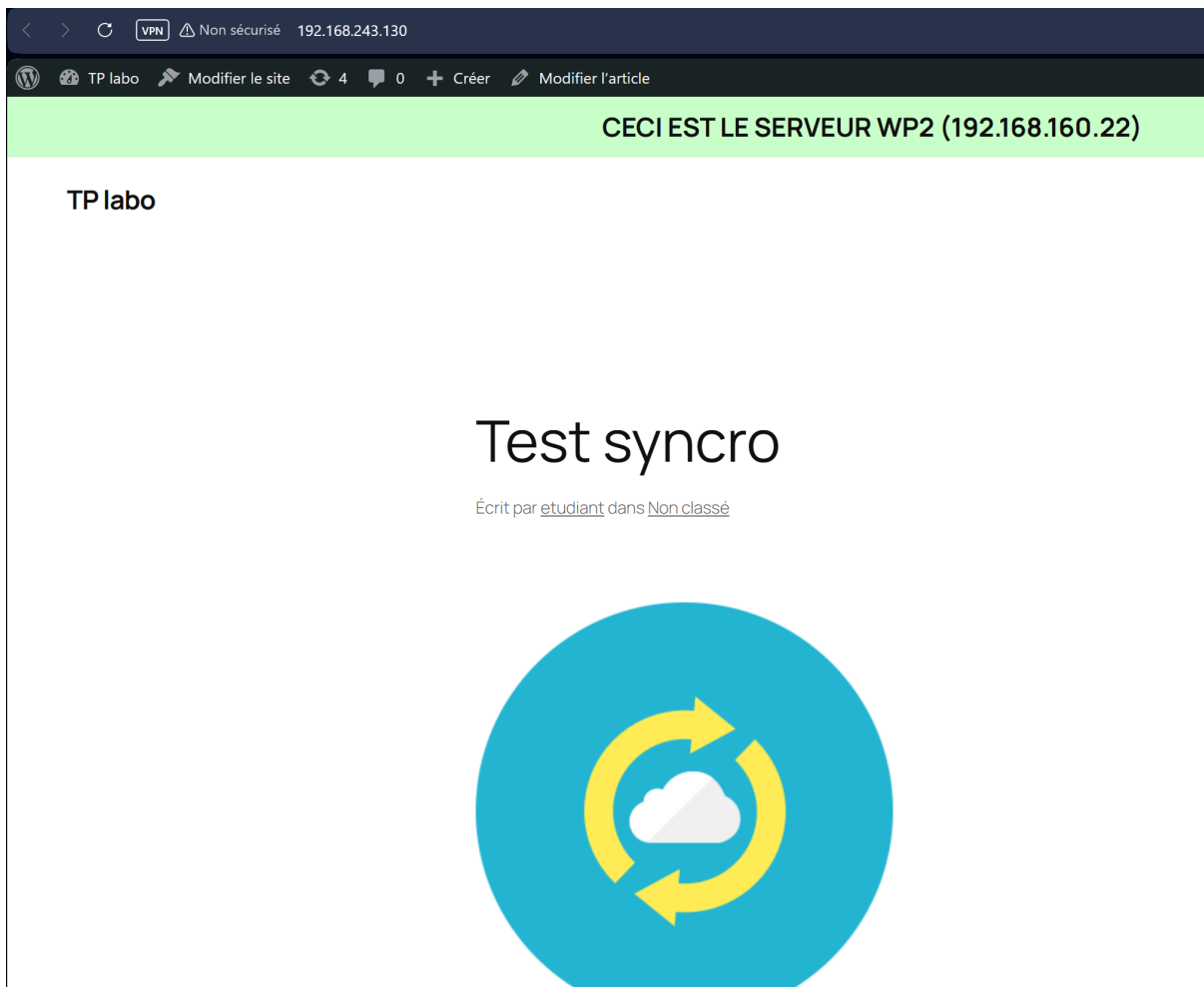
On réactive nginx sur WP2 :

TP labo

Test syncro

Écrit par [etudiant](#) dans [Non classé](#)





La synchronisation de la BDD fonctionne parfaitement.

5. Haute Disponibilité (HA) des adresses IP (Front et BDD)

Concept : L'adresse IP Virtuelle (VIP) Pour éviter de devoir reconfigurer de manière statique les composants de l'infrastructure en cas de panne, nous implémentons des adresses IP Virtuelles (VIP). Ces adresses flottantes sont hébergées par un cluster de serveurs mais ne pointent que vers un seul nœud actif à la fois. Si ce nœud défaille, l'IP bascule automatiquement vers le nœud de secours de manière transparente pour les clients et l'application.

Service utilisé : Keepalived (Protocole VRRP) Le paquet `keepalived` est déployé pour orchestrer cette redondance en s'appuyant sur le protocole standard VRRP (Virtual Router Redundancy Protocol). Il surveille en continu la disponibilité des machines et des services associés grâce à des scripts de tracking.

5.1. Implémentation de la VIP sur la couche Base de Données

1. Sécurisation et Prérequis Système (Sur BDD1 et BDD2) Après l'installation du paquet via `apt install keepalived`, deux actions de durcissement de sécurité sont indispensables pour se conformer aux exigences de sécurité du service :

- Restriction des droits d'accès au fichier de configuration (contenant le mot de passe de synchronisation en clair) : `chmod 600 /etc/keepalived/keepalived.conf`
- Création d'un utilisateur système dédié, sans privilèges et sans shell, pour l'exécution des scripts de vérification : `useradd -U -M -s /sbin/nologin keepalived_script`

2. Configuration du Nœud Primaire (BDD1 - MASTER) Le fichier `/etc/keepalived/keepalived.conf` est configuré pour accorder la priorité par défaut à BDD1, tout en surveillant activement le processus MariaDB :

```
root@mod-debian13:~# cat /etc/keepalived/keepalived.conf
# Paramètres généraux
global_defs {
    enable_script_security
}

# MariaDB est-il en cours d'exécution ?
vrrp_script check_mariadb {
    script "/usr/bin/pgrep mariadb"
    interval 1
    fall 2
    rise 2
}

# Configuration interface virtuelle
vrrp_instance VirtualIP {
    interface ens33
    state MASTER
    priority 100
    virtual_router_id 1
    authentication {
        auth_type PASS
        auth_pass Iut_2024
    }
    virtual_ipaddress {
        192.168.160.30
    }
    track_script {
        check_mariadb
    }
}
```

```

root@mod-debian13:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:66:ce:42 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c2966ce42
    inet 192.168.160.31/24 brd 192.168.160.255 scope global ens33
        valid_lft forever preferred_lft forever
    inet 192.168.160.30/32 scope global proto keepalived ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe66:ce42/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever

```

Explication de l'importance et de l'utilité des blocs de configuration :

- **Le bloc `global_defs`** : Il gère la sécurité globale du service Keepalived. Les paramètres `enable_script_security` et `script_user` forcent l'exécution des scripts de vérification avec un utilisateur aux droits restreints (et non root), évitant ainsi une faille de sécurité majeure sur le serveur.
- **Le bloc `vrp_script_check_mariadb`** : C'est le moniteur de santé (Health Check). Il définit la commande à lancer (`pgrep mariadb`) pour vérifier si la base de données fonctionne. Il définit aussi le rythme de vérification (`interval 1` seconde) et les seuils de tolérance (`fall 2` pour déclarer une panne après 2 échecs, `rise 2` pour le redéclarer sain après 2 succès).
- **Le bloc `vrp_instance VirtualIP`** : C'est le cœur de la Haute Disponibilité. Il définit les règles de l'élection VRRP : sur quelle interface écouter (`ens192`), quel est le rôle par défaut (`MASTER` ou `BACKUP`), la puissance de vote (`priority`), la clé de sécurité pour éviter l'usurpation (`authentication`) et l'IP flottante à attribuer (`virtual_ipaddress`). Enfin, l'option `track_script` permet de lier cette instance réseau à l'état de santé de MariaDB surveillé par le bloc précédent.

3. Configuration du Nœud Secondaire (BDD2 - BACKUP) Sur BDD2, le fichier est identique à l'exception du rôle initial et de la priorité réseau qui est abaissée pour s'effacer devant le MASTER :

- `state BACKUP`
- `priority 50`

```
root@mod-debian13:~# cat /etc/keepalived/keepalived.conf
# Paramètres généraux
global_defs {
    enable_script_security
}

# MariaDB est-il en cours d'exécution ?
vrrp_script check_mariadb {
    script "/usr/bin/pgrep mariadb"
    interval 1
    fall 2
    rise 2
}

# Configuration interface virtuelle
vrrp_instance VirtualIP {
    interface ens33
    state BACKUP
    priority 50
    virtual_router_id 1
    authentication {
        auth_type PASS
        auth_pass Iut_2024
    }
    virtual_ipaddress {
        192.168.160.30
    }
    track_script {
        check_mariadb
    }
}
```

4. Adaptation de l'application WordPress Pour que la haute disponibilité soit effective, la configuration de WordPress sur les serveurs web **wp1** et **wp2** doit être modifiée. Dans le fichier `wp-config.php`, la directive `DB_HOST` ne pointe plus vers l'IP physique de BDD1, mais vers la VIP :

```
GNU nano 8.4 /var/www/html/wordpress/wp-config.php *
/** Database username */
define( 'DB_USER', 'wp_admin' );

/** Database password */
define( 'DB_PASSWORD', 'Etudiant_1234' );

/** Database hostname */
define( 'DB_HOST', '192.168.160.30' );

/** Database charset to use in creating database tables. */
define( 'DB_CHARSET', 'utf8mb4' );

/** The database collate type. Don't change this if in doubt. */
define( 'DB_COLLATE', '' );
```

5.2. Tests et Validation du Failover

Test 1 : Validation nominale Après démarrage du service (`service keepalived restart`), la commande `ip a` permet de valider que la VIP `192.168.160.30` est bien assignée à l'interface `ens32` de BDD1, et reste invisible sur BDD2.

Test 2 : Simulation de panne de service (MariaDB)

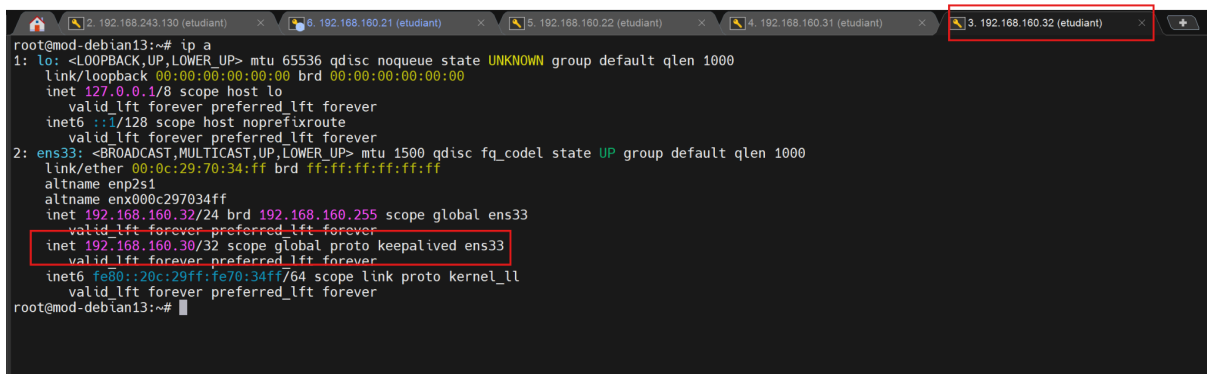
- **Action** : Arrêt volontaire du moteur de base de données sur le maître : `systemctl stop mariadb`.
- **Comportement attendu** : Le script `check_mariadb` échoue deux fois de suite (`fail 2`). Keepalived rétrograde le nœud BDD1. BDD2 ne recevant plus d'annonces VRRP prioritaires, il s'attribue immédiatement la VIP `192.168.160.30`.
- **Vérification de l'intégrité** : Un ping continu initié depuis un serveur WordPress vers la VIP ne subit aucune interruption (ou une perte maximale d'un seul paquet). Le site WordPress reste pleinement fonctionnel en lecture et en écriture, prouvant la réussite de la bascule transparente et le maintien de l'intégrité du jeu de données.

```

64 bytes from 192.168.160.30: icmp_seq=12 ttl=64 time=0.397 ms
64 bytes from 192.168.160.30: icmp_seq=13 ttl=64 time=0.564 ms
64 bytes from 192.168.160.30: icmp_seq=14 ttl=64 time=0.424 ms
64 bytes from 192.168.160.30: icmp_seq=15 ttl=64 time=0.750 ms
64 bytes from 192.168.160.30: icmp_seq=16 ttl=64 time=1.16 ms
64 bytes from 192.168.160.30: icmp_seq=17 ttl=64 time=1.29 ms
64 bytes from 192.168.160.30: icmp_seq=18 ttl=64 time=0.698 ms
64 bytes from 192.168.160.30: icmp_seq=19 ttl=64 time=0.791 ms
64 bytes from 192.168.160.30: icmp_seq=20 ttl=64 time=0.728 ms
64 bytes from 192.168.160.30: icmp_seq=21 ttl=64 time=0.799 ms
64 bytes from 192.168.160.30: icmp_seq=22 ttl=64 time=0.894 ms
64 bytes from 192.168.160.30: icmp_seq=23 ttl=64 time=0.916 ms
64 bytes from 192.168.160.30: icmp_seq=24 ttl=64 time=0.645 ms
64 bytes from 192.168.160.30: icmp_seq=25 ttl=64 time=0.503 ms
64 bytes from 192.168.160.30: icmp_seq=26 ttl=64 time=0.702 ms
64 bytes from 192.168.160.30: icmp_seq=27 ttl=64 time=0.782 ms
64 bytes from 192.168.160.30: icmp_seq=28 ttl=64 time=0.888 ms
64 bytes from 192.168.160.30: icmp_seq=29 ttl=64 time=0.791 ms
64 bytes from 192.168.160.30: icmp_seq=30 ttl=64 time=0.980 ms
64 bytes from 192.168.160.30: icmp_seq=31 ttl=64 time=1.13 ms
64 bytes from 192.168.160.30: icmp_seq=32 ttl=64 time=1.41 ms
64 bytes from 192.168.160.30: icmp_seq=33 ttl=64 time=0.651 ms
64 bytes from 192.168.160.30: icmp_seq=34 ttl=64 time=0.659 ms
64 bytes from 192.168.160.30: icmp_seq=36 ttl=64 time=1.97 ms
64 bytes from 192.168.160.30: icmp_seq=37 ttl=64 time=2.55 ms
64 bytes from 192.168.160.30: icmp_seq=38 ttl=64 time=0.252 ms
64 bytes from 192.168.160.30: icmp_seq=39 ttl=64 time=0.327 ms
64 bytes from 192.168.160.30: icmp_seq=40 ttl=64 time=0.264 ms
64 bytes from 192.168.160.30: icmp_seq=41 ttl=64 time=0.261 ms
64 bytes from 192.168.160.30: icmp_seq=42 ttl=64 time=0.767 ms
64 bytes from 192.168.160.30: icmp_seq=43 ttl=64 time=0.632 ms
64 bytes from 192.168.160.30: icmp_seq=44 ttl=64 time=0.952 ms
64 bytes from 192.168.160.30: icmp_seq=45 ttl=64 time=0.767 ms
64 bytes from 192.168.160.30: icmp_seq=46 ttl=64 time=0.893 ms
64 bytes from 192.168.160.30: icmp_seq=47 ttl=64 time=0.529 ms
64 bytes from 192.168.160.30: icmp_seq=48 ttl=64 time=1.16 ms

```

le ping est resté continue une micro hausse quand le mariadb de BDD1 à été coupé mais le ping a continuer.



```

root@mod-debian13:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:70:34:ff brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx00c297034ff
    inet 192.168.160.32/24 brd 192.168.160.255 scope global ens33
        valid_lft forever preferred_lft forever
    inet 192.168.160.30/32 scope global proto keepalived ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe70:34ff/64 scope link proto kernel_l1
        valid_lft forever preferred_lft forever
root@mod-debian13:~#

```

l'adresse en .30 est bien apparu sur BDD2.

5.2. Implémentation de la VIP sur la couche Frontend

Une logique rigoureusement identique est appliquée sur les deux serveurs Frontaux pour assurer la résilience du point d'entrée unique de la plateforme.

- **Script de tracking** : Surveillance du serveur web frontal via la commande `/usr/bin/pgrep nginx`.
- **Configuration VRRP** : Configuration d'une seconde instance VRRP distincte (avec un `virtual_router_id` unique, par exemple 2) pour gérer une VIP dédiée au trafic HTTP/HTTPS des utilisateurs (ex: `192.168.160.10`).
- **Rôles** : FRONT1 est défini comme **MASTER** (priorité 100) et FRONT2 comme **BACKUP** (priorité 50).

Config front1 :

```
GNU nano 8.4 /etc/keepalived/keepalived.conf *
global_defs {
    enable_script_security
    script_user keepalived_script
}

vrrp_script check_nginx {
    script "/usr/bin/pgrep nginx"
    interval 1
    fall 2
    rise 2
}

vrrp_instance VI_FRONT {
    interface ens33
    state MASTER
    priority 100
    virtual_router_id 2
    authentication {
        auth_type PASS
        auth_pass Iut_Front
    }
    virtual_ipaddress {
        192.168.160.10/24
    }
    track_script {
        check_nginx
    }
}
```

Conf front2 :

```
GNU nano 8.4 /etc/keepalived/keepalived.conf *
global_defs {
    enable_script_security
    script_user keepalived_script
}

vrrp_script check_nginx {
    script "/usr/bin/pgrep nginx"
    interval 1
    fall 2
    rise 2
}

vrrp_instance VI_FRONT {
    interface ens33
    state BACKUP
    priority 50
    virtual_router_id 2
    authentication {
        auth_type PASS
        auth_pass Iut_Front
    }
    virtual_ipaddress {
        192.168.160.10/24
    }
    track_script {
        check_nginx
    }
}
```

IP a ok sur front1 :

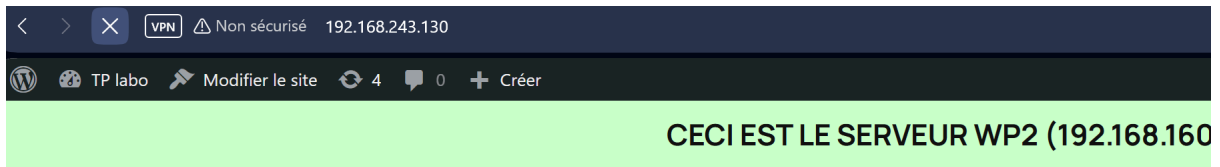
```
2. 192.168.243.130 (etud... x 7. 192.168.243.134 (etud... x 6. 192.168.160.21 (etudi... x 5. 192.168.160.22 (etudi... x 4. 192.168.160.23 (etudi... x
Réception de : 1 http://security.debian.org/debian-security trixie-security InRelease [43,4 kB]
Réception de : 2 http://deb.debian.org/debian trixie InRelease [140 kB]
Réception de : 3 http://deb.debian.org/debian trixie-updates InRelease [47,3 kB]
Réception de : 4 http://security.debian.org/debian-security trixie-security/main Sources [157 kB]
Réception de : 5 http://deb.debian.org/debian trixie/main Sources [10,5 MB]
Réception de : 6 http://security.debian.org/debian-security trixie-security/main amd64 Packages [171 kB]
Réception de : 7 http://security.debian.org/debian-security trixie-security/main Translation-en [108 kB]
Réception de : 8 http://deb.debian.org/debian trixie/main amd64 Packages [9 671 kB]
Réception de : 9 http://deb.debian.org/debian trixie/main Translation-en [6 485 kB]
27,4 Mo réceptionnés en 2s (14,6 Mo/s)
89 paquets peuvent être mis à jour. Exécutez « apt list --upgradable » pour les voir.
Notification : Le dépôt « http://deb.debian.org/debian trixie InRelease » a modifié sa valeur « Version » de 1:2.3.3-1 à 1:2.3.3-1.
keepalived est déjà la version la plus récente (1:2.3.3-1).
Sommaire :
Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jour : 89
root@mod-debian13:~# nano /etc/keepalived/keepalived.conf
root@mod-debian13:~#
root@mod-debian13:~#
root@mod-debian13:~# chmod 600 /etc/keepalived/keepalived.conf
systemctl restart keepalived
root@mod-debian13:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 1000
    link/ether 00:0c:29:8a:37:50 brd ff:ff:ff:ff:ff:ff
    altname enp2s2
    altname enx000c298a3750
    inet 192.168.160.20/24 brd 192.168.160.255 scope global ens34
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe8a:3750/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
3: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:8a:37:46 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c298a3746
    inet 192.168.243.130/24 brd 192.168.243.255 scope global dynamic noprefixroute ens33
        valid_lft 1259sec preferred_lft 1034sec
    inet 192.168.160.10/24 scope global proto keepalived ens33
        valid_lft forever preferred_lft forever
    inet6 fe80::a17a:ae72:5ed:327c/64 scope link
        valid_lft forever preferred_lft forever
root@mod-debian13:~#
```

Test de fonctionnement le même que pour les BDD:

Sur front1:

```
2. 192.168.243.130 (etud... 7. 192.168.243.134 (etud... 6. 192.168.160.21 (etud...
Réception de : 2 http://deb.debian.org/debian trixie InRelease [140 kB]
Réception de : 3 http://deb.debian.org/debian trixie-updates InRelease
Réception de : 4 http://security.debian.org/debian-security trixie-se
Réception de : 5 http://deb.debian.org/debian trixie/main Sources [10,
Réception de : 6 http://security.debian.org/debian-security trixie-se
Réception de : 7 http://security.debian.org/debian-security trixie-se
Réception de : 8 http://deb.debian.org/debian trixie/main amd64 Packag
Réception de : 9 http://deb.debian.org/debian trixie/main Translation-
27,4 Mo réceptionnés en 2s (14,6 Mo/s)
89 paquets peuvent être mis à jour. Exécutez « apt list --upgradable »
Notification : Le dépôt « http://deb.debian.org/debian trixie InReleas
keepalived est déjà la version la plus récente (1:2.3.3-1).
Sommaire :
Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jou
root@mod-debian13:~# nano /etc/keepalived/keepalived.conf
root@mod-debian13:~#
root@mod-debian13:~#
root@mod-debian13:~# chmod 600 /etc/keepalived/keepalived.conf
systemctl restart keepalived
root@mod-debian13:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN gr
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.0.1/8 scope host lo
valid_lft forever preferred_lft forever
inet6 ::1/128 scope host noprefixroute
valid_lft forever preferred_lft forever
2: ens34: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel st
link/ether 00:0c:29:8a:37:50 brd ff:ff:ff:ff:ff:ff
altname enp2s2
altname enx000c298a3750
inet 192.168.160.20/24 brd 192.168.160.255 scope global ens34
valid_lft forever preferred_lft forever
inet6 fe80::20c:29ff:fe8a:3750/64 scope link proto kernel_ll
valid_lft forever preferred_lft forever
3: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel st
link/ether 00:0c:29:8a:37:46 brd ff:ff:ff:ff:ff:ff
altname enp2s1
altname enx000c298a3746
inet 192.168.243.130/24 brd 192.168.243.255 scope global dynamic n
valid_lft 1259sec preferred_lft 1034sec
inet 192.168.160.10/24 scope global proto keepalived ens33
valid_lft forever preferred_lft forever
inet6 fe80::a17a:ae72:5ed:327c/64 scope link
valid_lft forever preferred_lft forever
root@mod-debian13:~# systemctl stop nginx
root@mod-debian13:~#
```

Le site est toujours fonctionnelle grace au front2:



TP labo

Blog

Test syncro

